Eidonic Core Technical Stack Specification v1

A phased technical specification for building the governed core, using ECP as the sovereign runtime envelope and local models as the initial intelligence substrate.

Status	Working draft for build execution
Version	v1.0
Intended Use	Implementation guide tied to the ruthless build map
Primary Position	Build the governed core loop first. Treat manifestations as downstream proofs, not as the substrate.
Sovereign Runtime	ECP as runtime envelope, policy membrane, provenance shell, and bundle lifecycle manager

Build doctrine: ECP replaces Docker as the control philosophy and execution membrane. It does not need to recreate the entire container industry in phase one. Use hardened lower substrate only where it saves years and does not violate the organism's law.

1. Executive position

This specification converts the ruthless build map into a concrete technical sequence. It names the stack, the service boundaries, the runtime posture, the local model plan, and the order in which each layer should be built.

The central decision is simple: build the governed core first. Do not begin with the broad Eidonic universe, and do not confuse symbolic richness with runtime maturity.

1.1 What v1 is trying to prove

- A human request can enter the system as a typed signal, pass through thresholding, routing, bounded orchestration, memory posture, witness, and return.
- ECP can serve as the sovereign runtime envelope for the core without requiring a full reimplementation of all container primitives in phase one.
- Local models can bootstrap the core loop, enabling orchestration, review, retrieval, and disciplined memory before any discussion of training from scratch.
- The visible interface can tell the truth about state, provenance, confidence, and commit posture rather than act as decorative theater.

1.2 Non-goals for v1

- Do not build a general autonomous swarm.
- Do not build the full embodied universe, sanctuary stack, energy stack, robotics stack, and spatial shells in parallel.
- Do not treat every EKRP as a separate model or microcompany.
- Do not rebuild cluster orchestration, OCI image ecosystems, or kernel-level isolation before the core loop itself is proven.

2. Grounding assumptions

This spec is grounded in the existing Eidonic Core architecture and in the published ECP community release.

Grounding source	What it contributes	Why it matters for this spec
Living System Architecture	Organism model, layer distinctions, v1 build path	Prevents category collapse. The stack must preserve threshold, memory, witness, routing, interface, and governance as distinct functions.
Data Metabolism	Typed state model for transformation	Supplies the signal-to-memory lifecycle and the minimal object vocabulary that the runtime must support.
Memory Fabric	Continuity, consent, provenance, release	Forces memory architecture to be layered, reviewable, and non-

Grounding source	What it contributes	Why it matters for this spec
		possessive.
Nervous System	Routing, reflex, weave, projection, return	Defines the service map for ingress, routing, eventing, and recalibration.
Interface and Anatomy Dashboard	Truthful surface and operator modes	Prevents the UI from becoming an atmospheric lie.
ECP Community v0.1.0	Bundle declaration, sealing, signing, verify/run gate, FastAPI, Vite React GUI	Anchors this spec in a real published runtime instead of a blank page.

3. Core architectural decisions

3.1 Decision A: ECP is sovereign

ECP is the primary runtime envelope. It owns bundle identity, manifest lifecycle, integrity verification, run gating, policy attachment, and provenance wrapping.

In v1, ECP is not required to replace every lower-level execution primitive in existence. It must replace Docker as the governing model and the launch membrane. That is enough to be real.

3.2 Decision B: local models before custom training

Use local models as the initial cognitive substrate. The bottleneck is not raw model ownership. The bottleneck is orchestration discipline, memory law, witness, and review. Training from scratch can wait until the runtime has a body.

3.3 Decision C: typed records before lore growth

Every important movement through the system should produce typed records that can be stored, traced, reviewed, or released. The organism becomes real when its flows are visible in data structures.

3.4 Decision D: smallest sufficient path

Routing must always favor the smallest sufficient path. Reflexes should stay reflexive. One-EKRP tasks should not wake a council. Multi-organ weaving should be earned.

3.5 Decision E: manifestation follows substrate

Thought Veil, Thought Projection, SOP, VR Studio, sanctuary systems, and downstream world systems are not phase-one substrate. They are later proofs that the substrate can move without lying.

4. ECP position in the stack

The published ECP community release already gives you a real starting point: an edc.bundle.json declaration, manifest sealing with SHA3-256, Ed25519 signing and verification, a pre-run integrity gate, a FastAPI backend, a Vite React GUI, and a desktop-oriented vessel workflow. It is explicitly container-less in the community form and explicitly does not yet include OS-level isolation. The technical stack must respect those truths instead of pretending they are already solved.

4.1 What ECP must do in v1

- Start and stop core services under a named bundle or vessel identity.
- Bind manifests, signatures, versions, and runtime policy to every launched service group.
- Enforce pre-run integrity checks and signature posture before privileged core services come up.
- Issue runtime identities and trace tokens that all services must carry into event, memory, and witness records.
- Own bundle manifests for local models, model adapters, interface shells, and core services.
- Provide a stable place to attach future filesystem, network, process, and memory policy gates.

4.2 What ECP should not attempt in v1

- Do not recreate OCI image registries.
- Do not build a full Kubernetes replacement.
- Do not promise mature OS sandboxes before they exist.
- Do not hide the community limitation that launched processes still run with user privileges.

4.3 ECP evolution path

Layer	Phase	ECP role	Notes
Base	0 to 1	Bundle launch, manifest, sign/verify, run gate	Uses current community posture as the trusted seed.
Control	2	Runtime identity, service lifecycle, policy attachment	Binds Guardian and Ravien to launch-time truth.
Envelope	3 to 4	Typed model bundles, registry manifests, reviewable update flow	Makes model and service upgrades witnessable.
Hardening	5+	Filesystem/network/ process policy gates and stronger isolation	Only after the core loop works and the policy model is mature.

5. Phase-by-phase technical stack

The order below is the order of construction, not merely the order of description. It is intentionally unforgiving.

Phase	Name	Primary proof	Core technologies	Exit gate
0	Substrate	Single-node local core can boot reliably	Python 3.12, Node 20, FastAPI, Vite React, Postgres, NATS, Qdrant, ECP	One-command bring-up, typed health, signed bundles
1	Threshold and Session	One request can be normalized, thresholded, and bound to session	Signal gateway, Herald, Session Engine, Eidon Orchestrator, Ollama	Request enters and returns with explicit state
2	Governance and Witness	Consequential paths produce reviewable truth	Guardian, Ravien Witness, policy registry, review engine	Every non-trivial action yields provenance and review posture
3	Organ Activation	Smallest-sufficient orchestration actually works	EKRP registry, capability graph, reflex router, SOP weaver	One-EKRP and multi-EKRP paths both behave predictably
4	Memory and Metabolism	Continuity and release become governed	Metabolism engine, memory fabric, retrieval broker, Neo4j optional	System can store, recall, review, renew, and release
5	Truthful Surface	Operator can see the body without deception	Dashboard, websocket/event views, provenance panels	State, confidence, preview, commit, and drift are visible
6	Projection and Embodiment	Core can project into richer shells without losing law	Projection adapters, preview shell, spatial bridge	At least one manifestation channel works without bypassing witness

Phase 0: Substrate and local development spine

The purpose of phase zero is to stop debating basics and create a reproducible local machine on which every later claim rests.

What to use

- Ubuntu or WSL for the primary developer path
- Python 3.12+ for orchestration and service code
- Node 20+ for the dashboard and any vessel UI work
- FastAPI for typed service APIs
- PostgreSQL for canonical records and system truth
- NATS for the event bus and request-reply patterns
- Qdrant for embeddings and filtered semantic retrieval
- Object storage for blobs and exported artifacts

What to build

- ecp-supervisor
- bootstrap scripts
- service manifests and signing flow
- local model adapter bundle
- health and diagnostics endpoints

Why this phase exists

- Without a reproducible substrate, every later discussion turns into mood and machine drift.

Phase 1: Threshold and session spine

This phase proves ingress, pacing, and living-session binding. It is the first real body.

What to use

- signal-gateway service
- Herald threshold service
- Session Engine
- Eidon Orchestrator
- Ollama for local-model serving during the bootstrap period

Initial model pattern

- One strong local instruct model for general orchestration
- One smaller local instruct model for reflexes and threshold prompts
- One embedding model for retrieval

Key scripts

- bootstrap_ecp.py
- run_local_model_stack.py
- register_signal_schemas.py
- smoke_test_core_loop.py

Phase 2: Governance and provenance spine

This phase turns the system from responsive to reviewable.

What to use

- Guardian policy engine
- Ravien witness service
- policy registry
- review engine
- signed runtime identity propagation

Primary data stores

- PostgreSQL for witness, review, sessions, and policy records

- NATS for event flow
- Qdrant only for retrieval support, not as canonical truth

What must become true

- No meaningful state change happens without a provenance trail.
- The system can say what changed, why, under what policy, and with what confidence.

Phase 3: Organ activation and controlled weaving

This phase introduces EKRPs as bounded capabilities rather than theatrical personalities.

What to use

- EKRP registry
- capability graph
- reflex router
- single-organ path
- SOP weaver for multi-organ sessions

Design law

- Differentiate organs by instruction pack, tool access, retrieval scope, memory access class, and review requirements before you differentiate them by separate model weights.

Do not do this

- Do not assign a bespoke model to every EKRP. Do not let every query wake a council.

Phase 4: Memory fabric and data metabolism

This phase gives the body continuity and restraint.

What to use

- metabolism engine
- memory-fabric service
- retrieval broker
- release engine
- Neo4j only if relation depth actually justifies it

Canonical object classes

- SignalEvent
- IntentPacket
- ReflectionRecord
- IntegrationProposal
- WitnessSeal
- ArchiveRecord
- ReleaseRecord

Retrieval order

- Canonical facts from PostgreSQL
- semantic candidates from Qdrant
- relationship trails from Neo4j when needed
- policy-aware merge before any answer or projection

Phase 5: Truthful interface and anatomy dashboard

Now, and only now, the visible body can be built without fraud.

What to use

- TypeScript dashboard
- role-aware API gateway
- websocket or event-stream bridge
- provenance trace views
- state and drift components

Must-have surfaces

- overview
- sessions
- metabolism
- memory
- nervous system
- governance
- provenance

UI law

- Always show preview versus commit, confidence versus certainty, witness versus unwitnessed, and degradation versus silent failure.

Phase 6: Projection and embodiment pathways

This phase begins manifestation, but it still remains subordinate to the core law.

What to use

- projection adapter
- preview shell
- commit gate
- spatial session bridge
- manifest-bound channel adapters

What qualifies as success

- At least one external surface can receive outputs or previews without bypassing threshold, witness, or return.

What remains prohibited

- No silent autonomous outward action. No bypass around Guardian or Ravien.

6. Service catalog

The following services should exist as named units, whether they begin as processes, ECP bundles, or later become more isolated vessels.

Service	Phase	Primary language	State	Purpose
ecp-supervisor	0	Python	light	launch control, manifests, identity, lifecycle
signal-gateway	1	Python	light	normalize chat, files, commands into typed signals
herald-service	1	Python	light	thresholding, clarification, sensitivity posture
session-engine	1	Python	medium	session identity, mode, context budget, closure
eidon-orchestrator	1	Python	medium	conscious framing, route proposal, final return shaping
guardian-engine	2	Python	medium	policy checks on ingress, routes, writes, and projections
ravien-witness	2	Python	medium	provenance, witness seals, correction lineage
review-engine	2	Python	medium	review packets, dispositions, escalation
ekrp-registry	3	Python	light	organ definitions, boundaries, capability metadata
capability-graph	3	Python	medium	map intents to organs, tools, policies, model classes
sop-weaver	3	Python	medium	coordinate multi-organ sessions
metabolism-engine	4	Python	medium	typed movement through reflect, dream, relearn, integrate
memory-fabric	4	Python	heavy	continuity objects, recall, renewal,

Service	Phase	Primary language	State	Purpose
				release
retrieval-broker	4	Python	medium	policy-aware multi-store retrieval
dashboard-api	5	Python	light	operator-facing access and live state feeds
dashboard-ui	5	TypeScript	none	truthful surface and operator modes
projection-adapter	6	Python or TS	light	bridge outputs into later manifestation channels

7. Local model strategy

Local models are the sane starting substrate because they let you prove orchestration and data law without depending on rented cognition. They are not the final doctrine. They are the initial foundry.

Band	Role	Recommended use	Notes
A	Reflex	Small fast instruct model for acknowledgements, clarification, light threshold prompts, routing hints	Cheap, disposable, never canon-bearing by itself
B	Main orchestration	Stronger instruct model for Eidon, Herald, and most single-organ work	Default general engine in early build
C	Deep synthesis	Same stronger model or a heavier local model for Ravien and difficult weave paths	Use sparingly and always under witness
E	Embeddings	Dedicated local embedding model for Qdrant collections and recall	Retrieval support, not truth
V	Vision	Later local vision-capable model for file or image ingress	Not needed for phase one unless image-heavy workflows matter

7.1 Serving progression

- Start with Ollama as the local serving layer because it is easy to stand up and good enough for bootstrapping local instruct, embedding, and later vision models.
- Move to vLLM when concurrency, server controls, or structured output reliability become important enough to justify the added complexity.

- Do not let model-serving complexity outrun the runtime. Serving should support the organism, not become the organism.

8. Data architecture and typed records

PostgreSQL is the canonical truth store. Qdrant is for semantic proximity and filtered vector recall. Neo4j is optional and should be introduced only when constellation and relation queries become structurally painful in plain relational form.

Store	Purpose	Phase	What belongs there
PostgreSQL	Canonical system records	0 onward	sessions, signals, witness, policy, review, memory object metadata
NATS	Event bus and request-reply	0 onward	typed events, control signals, live updates, durable streams where needed
Qdrant	Semantic retrieval layer	0 onward	embeddings, payload-filtered memory candidates, retrieval indexes
Neo4j	Relational memory graph	4 onward	thread links, constellation graphs, lineage trails when joins become insufficient
Object storage	Blob layer	0 onward	documents, exports, artifacts, manifests, generated media, large attachments

8.1 Mandatory typed records for v1

1. SignalEvent
2. IntentPacket
3. ReflectionRecord
4. IntegrationProposal
5. WitnessSeal
6. ArchiveRecord
7. ReleaseRecord

Nothing consequential should pass through the core as anonymous text blobs once the first loop exists.

9. Security and trust posture

The security story must remain honest. Community ECP is integrity-aware and verifiable, but it is not yet a mature sandbox. This spec therefore separates trust layers instead of pretending one layer solves all others.

Layer	What v1 can truthfully claim	What remains future hardening
Integrity	Bundles can be sealed, signed, verified, and blocked from running if the manifest check fails	Signature enforcement on all paths and richer attestations
Policy	Guardian rules can bind routes, writes, tools, and projections at application level	OS-level enforcement for filesystem, process, and network
Provenance	Ravien can witness actions, corrections, and review lineage	tamper-resistant audit chaining and external attestations
Isolation	Limited to user/process boundaries in the community posture	real OS sandboxes, cgroups, job objects, and stricter host controls

10. Recommended implementation sequence

This is the order in which work should be staffed and executed.

Wave	Duration bias	Primary outputs	Dependencies	Kill question
W1	short	ECP bootstrap, Postgres, NATS, Qdrant, health endpoints	none	Can one command bring the core up reliably?
W2	short	Signal gateway, Herald, Session Engine, Eidon, local models	W1	Can one request move through the body and return?
W3	medium	Guardian, Ravien, witness schema, review flow	W2	Can the system explain itself after acting?
W4	medium	EKRP registry, capability graph, single-organ and weave paths	W3	Can the smallest sufficient route be selected and enforced?
W5	medium	Metabolism, memory fabric, retrieval broker, release engine	W3	Can memory exist without hoarding or false permanence?
W6	short	Dashboard surfaces and provenance views	W2 to W5	Can an operator see the truth of the system?
W7	selective	Projection adapter and one manifestation proof	W2 to W6	Can the core touch an external surface without bypassing

Wave	Duration bias	Primary outputs	Dependencies	Kill question
				law?

11. Suggested repository layout

```
eidon/
  ecp/
    supervisor/
    manifests/
    identity/
    policy/
    provenance/
  core/
    signal_gateway/
    herald_service/
    session_engine/
    eidon_orchestrator/
    guardian_engine/
    ravien_witness/
    review_engine/
    ekrp_registry/
    capability_graph/
    sop_weaver/
    metabolism_engine/
    memory_fabric/
    retrieval_broker/
    release_engine/
  models/
    ollama_adapter/
    vllm_adapter/
    embedding_service/
    prompt_packs/
  interface/
    dashboard_api/
    dashboard_ui/
  scripts/
    bootstrap_ecp.py
    run_local_model_stack.py
    register_signal_schemas.py
    seed_policies.py
    smoke_test_core_loop.py
  infra/
    dev/
    manifests/
```

12. Definition of done for v1

- A signed ECP bundle can launch the core locally and fail honestly when integrity is broken.
- A user request can move through typed ingress, thresholding, routing, orchestration, witness, and return.
- At least one reflex path, one single-organ path, and one multi-organ weave path work under policy control.

- The system stores only permitted continuity objects and can release what should not persist.
- The dashboard reveals state, provenance, confidence, preview, and commit posture truthfully.
- At least one projection or manifestation path exists and still passes through Guardian, Ravien, and return.

12.1 What would count as failure

- If the system still cannot show provenance for meaningful actions, the architecture is still mostly theater.
- If memory permanence happens without explicit policy and witness, the organism has no law.
- If ECP becomes a branding layer with no effect on lifecycle, integrity, or identity, it has failed its role.
- If manifestations arrive before the governed core loop exists, sequencing has collapsed.

Appendix A. Grounding notes

This document is aligned to the Eidonic Core material already established in the project: the Living System Architecture, Data Metabolism, Memory Fabric, Nervous System, and Interface/Anatomy Dashboard. It is also grounded in the published ECP community release, which currently provides bundle declaration, sealing, signing, verification, run-gating, FastAPI, and a Vite React GUI, while explicitly not yet claiming OS-level isolation.

The stack choices in this document are recommendations for build execution. They are not all sacred. What is sacred is the order: governed ingress, session, witness, policy, memory law, truthful surface, then manifestation.

End of specification